

INTENSIVE DEEP LEARNING SYLLABUS

7-Day Advanced Review & Practical Implementation Plan | Modules 01–17

Duration: 7 Days • Recommended Study Load: 8–9 Hours/Day • Total: 56–63 Hours

Curriculum Area	Deep Learning, Neural Networks & PyTorch
Coverage	Modules 01–17
Format	Concepts + Mathematics + Implementation + Practice + Project
Learner Profile	Learner with existing foundational knowledge seeking detailed mastery
Primary Framework	PyTorch
Core Outcome	Ability to explain, derive, implement, train, evaluate and debug neural networks

1. Curriculum Purpose

This intensive syllabus consolidates Modules 01–17 into a structured seven-day learning pathway. Because the learner already has familiarity with the core concepts, instructional time is weighted toward mathematical reasoning, implementation, debugging, training dynamics and integrated project work rather than re-teaching introductory definitions.

2. Learning Outcomes

- Explain the architecture and learning mechanism of a perceptron and multilayer perceptron (MLP).
- Derive and apply gradient descent, loss functions, activation functions and parameter-update rules.
- Perform forward propagation and backpropagation using computational graphs and the chain rule.
- Use PyTorch tensors, automatic differentiation (autograd), nn.Module, Sequential and optimizers.
- Construct complete training pipelines including data loading, batching, evaluation and device management.
- Differentiate Batch Gradient Descent, Stochastic Gradient Descent and Mini-Batch Gradient Descent.
- Implement custom Dataset and DataLoader workflows and understand their key parameters.
- Select and explain appropriate regression and classification loss functions.
- Identify and explain vanishing-gradient behavior and common mitigation strategies.
- Build and evaluate an end-to-end ANN image-classification project.

3. Weekly Structure

Day	Modules	Primary Focus	Hours
Day 1	01–04	Perceptron, decision boundaries, learning rules, gradient descent	8–9
Day 2	05–06	Activation functions, losses, MLP, forward propagation	8–9
Day 3	08	Backpropagation, chain rule, computational	8–9

		graphs	
Day 4	09–10	PyTorch tensors, operations, autograd	8–9
Day 5	11–13	Training pipeline, nn.Module, optimizers, GD variants	8–9
Day 6	15–16	Dataset, DataLoader, evaluation, GPU workflow	8–9
Day 7	17 + Review	Loss functions, vanishing gradient, integrated project	8–9

Module 01: Introduction to Deep Learning and Perceptrons

Scope / Lesson Components

- 1.1 Introduction to the Course
- 1.2 Why Deep Learning?
- 1.3 Use Cases of Deep Learning
- 1.4 Machine Learning vs Deep Learning
- 1.5 How Machine Learning Fails
- 1.6 Biological Neuron
- 1.7 Artificial Neuron
- 1.8 Perceptron Architecture
- 1.9 Mini Project Using Perceptron
- 1.10 Module Summary
- Practice: perceptron fundamentals and architecture

Detailed Knowledge Areas

- Neuron model and analogy between biological and artificial neurons
- Inputs, weights, bias, weighted sum and activation
- Perceptron as a binary linear classifier
- Role of data representation and feature engineering
- Linear decision surfaces and limitations
- Practical perceptron implementation

Module 02: How Perceptrons Learn

Scope / Lesson Components

- 2.1 Introduction
- 2.2 Weight Updation Formula: Introduction
- 2.3 Weight Updation Formula: Intuition
- 2.4 Solving Logical AND Gate – Part 1
- 2.5 Solving Logical AND Gate – Part 2
- 2.6 Perceptron From Scratch – Part 1
- 2.7 Perceptron From Scratch – Part 2
- 2.8 OR and XOR Gates Using Perceptron
- 2.9 Why Perceptron Fails at XOR
- 2.10 Module Summary
- Quiz and practice problems

Detailed Knowledge Areas

- Perceptron learning rule and update mechanism
- Learning rate, error and parameter adjustment
- AND/OR gate implementation
- XOR and linear separability
- Perceptron from scratch in Python
- Edge cases and common implementation mistakes

Module 03: Decision Boundaries and Perceptron Learning

Scope / Lesson Components

- 3.1 Introduction
- 3.2 Decision Boundary and Bias Visualization
- 3.3 Perceptron Learning Rule – Part 1
- 3.4 Perceptron Learning Rule – Part 2
- 3.5 Perceptron Learning Rule – Part 3
- 3.6 TensorFlow Playground
- 3.7 Module Summary
- Practice and guided implementation

Detailed Knowledge Areas

- Decision boundary as a geometric representation of a classifier
- Effect of weights and bias on the boundary
- Learning rule in geometric terms
- Linearly separable vs non-separable datasets
- Interactive visualization and interpretation

Module 04: Gradient Descent to Loss Functions

Scope / Lesson Components

- 4.1 Introduction
- 4.2 Gradient Descent – Part 1
- 4.3 Gradient Descent – Part 2
- 4.4 Gradient Descent – Part 3
- 4.5 Perceptron Loss Function – Part 1
- 4.6 Perceptron Loss Function – Part 2
- Practice and implementation

Detailed Knowledge Areas

- Objective functions and optimization
- Derivative and gradient intuition
- Gradient descent update rule
- Learning rate and convergence
- Loss landscape and parameter optimization
- Perceptron loss and error measurement

Required Mastery

- Derive a gradient-descent update for a simple loss.
- Explain learning-rate effects using a loss curve.
- Implement one optimization example from scratch.

Module 05: Activation Functions and Loss

Scope / Lesson Components

- 5.1 Introduction
- 5.2 Why Activation Functions Are Needed
- 5.3 Sigmoid Activation Function
- 5.4 ReLU Activation Function
- 5.5 Softmax Activation Function – Part 1
- 5.6 Softmax Activation Function – Part 2
- 5.7 Binary Cross Entropy – Part 1
- 5.8 Binary Cross Entropy – Part 2
- 5.9 Advantages and Disadvantages of Perceptron
- 5.10 Module Summary
- Quiz and practice problems

Detailed Knowledge Areas

- Need for non-linearity in neural networks
- Sigmoid equation, range, derivative and saturation
- ReLU equation, derivative, advantages and dying-ReLU issue
- Softmax, logits and multiclass probability distribution
- Binary Cross Entropy and probabilistic interpretation
- Activation/loss selection for classification

Required Mastery

- Compute sigmoid, ReLU and softmax outputs.
- Explain saturation and dying ReLU.
- Calculate BCE for simple binary predictions.

Module 06: Multi-Layer Perceptrons and Forward Propagation

Scope / Lesson Components

- 6.1 Introduction
- 6.2 Idea of Multi-Layer Perceptron
- 6.3 Basic Idea of a Neural Network
- 6.4 MLP Notation – Part 1
- 6.5 MLP Notation – Part 2
- 6.6 Forward Propagation – Part 1
- 6.7 Forward Propagation – Part 2
- 6.8 Module Summary
- Quiz and practice problems

Detailed Knowledge Areas

- Input, hidden and output layers
- Neurons, connections and trainable parameters
- Layer-wise matrix notation
- Affine transformation $Z = WX + b$

- Activation $A = f(Z)$
- Multi-layer forward-pass calculations

Required Mastery

- Perform a complete two-layer forward pass numerically.
- Track tensor dimensions through every layer.

Module 07: Deep Learning Assignment 01

Scope / Lesson Components

- Exam instruction and assessment orientation
- 7.1 Assignment 01 Instructions
- DL Assignment 01
- Quiz/assessment preparation

Detailed Knowledge Areas

- Translate theoretical concepts into implementation
- Apply perceptron, optimization and neural-network concepts
- Produce reproducible code and interpret results

Module 08: Backpropagation Fundamentals

Scope / Lesson Components

- 8.1 Introduction
- 8.2 What Is Backpropagation?
- 8.3 Backpropagation Calculation – Part 1
- 8.4 Backpropagation Calculation – Part 2
- 8.5 Backpropagation Calculation – Part 3
- 8.6 Backpropagation Calculation – Part 4
- 8.7 Module Summary
- Practice and numerical problems

Detailed Knowledge Areas

- Forward pass, loss and backward pass
- Computational graphs
- Partial derivatives
- Chain rule
- Gradient propagation through layers
- Weight and bias gradients
- Manual numerical backpropagation

Required Mastery

- Derive gradients using the chain rule.
- Complete at least one full manual backpropagation problem.

Module 09: PyTorch Basics and Backpropagation Implementation

Scope / Lesson Components

- 9.1 Introduction
- 9.2 PyTorch Introduction
- 9.3 Creating Tensors
- 9.4 Tensor Shapes
- 9.5 Tensor Data Types
- 9.6 Scalar Operations
- 9.7 Element-Wise Operations
- 9.8 Reduction Operations
- 9.9 Matrix Operations
- 9.10 Comparison Operations
- 9.11 Special Functions
- 9.12 In-Place Operations
- 9.13 Copying a Tensor
- 9.14 Backpropagation Implementation Introduction
- 9.15 Weight Initialization
- 9.16 Forward Propagation Implementation
- 9.17 Weight Updation Implementation
- 9.18 Module Summary
- Quiz and practice problems

Detailed Knowledge Areas

- Tensor creation, shape and dtype
- Element-wise, reduction and matrix operations
- Comparison and special operations
- In-place operations and tensor copying
- Manual neural-network implementation in PyTorch
- Weight initialization, forward propagation and weight update

Module 10: Computation Graph and Autograd

Scope / Lesson Components

- 10.1 Introduction
- 10.2 Why Autograd?
- 10.3 Computation Graph
- 10.4 Basic Idea of Autograd
- 10.5 Autograd Using Real Example – Part 1
- 10.6 Autograd Using Real Example – Part 2
- 10.7 Autograd Using Real Example – Part 3
- 10.8 Autograd Using Real Example – Part 4
- 10.9 Autograd With Multiple Values
- 10.10 How to Clear Gradients
- 10.11 Module Summary

- Practice problems

Detailed Knowledge Areas

- Dynamic computation graphs
- `requires_grad` and gradient tracking
- `backward()` and gradient calculation
- Reading gradients through `.grad`
- Multiple variables and graph dependencies
- Gradient accumulation and clearing gradients

Required Mastery

- Explain how autograd constructs and traverses a computation graph.
- Use `backward()` and inspect gradients correctly.

Module 11: PyTorch Training Pipeline From Scratch

Scope / Lesson Components

- 11.1 This Week's Plan
- 11.2 Introduction
- 11.3 Data Loading and Basic Preprocessing
- 11.4 NumPy to Tensor Conversion
- 11.5 Defining Model
- 11.6 Training Pipeline
- 11.7 Model Evaluation
- 11.8 Module Summary
- Practice and implementation

Detailed Knowledge Areas

- Raw data to tensor pipeline
- Model definition
- Forward pass, loss, backward pass and optimization
- Epoch-based training
- Training and evaluation modes
- Basic performance measurement

Module 12: PyTorch `nn.Module` and `torch.optim`

Scope / Lesson Components

- 12.1 Introduction
- 12.2 PyTorch `nn.Module` Overview
- 12.3 Define Model Class – Part 1
- 12.4 Define Model Class – Part 2
- 12.5 Add More Layers
- 12.6 Sequential Container
- 12.7 Improving Training Pipelines

- 12.8 PyTorch torch.optim Module
- 12.9 torchinfo
- 12.10 Module Summary
- Quiz and practice

Detailed Knowledge Areas

- Object-oriented model design with nn.Module
- Custom forward methods
- Layer composition and Sequential
- Trainable parameters
- Optimizer configuration
- Model inspection and parameter summaries

Required Mastery

- Build a custom nn.Module and an equivalent Sequential model.
- Configure an optimizer and inspect model parameters.

Module 13: Training Dynamics – Batch, SGD and Mini-Batch

Scope / Lesson Components

- 13.1 Introduction
- 13.2 What Is Gradient Descent?
- 13.3 Batch Gradient Descent
- 13.4 Stochastic Gradient Descent
- 13.5 Batch GD vs Stochastic GD
- 13.6 Code Example
- 13.7 Mini-Batch Gradient Descent
- 13.8 Module Summary
- Practice

Detailed Knowledge Areas

- Gradient-estimation strategies
- Batch GD: full-dataset update
- SGD: single-sample update
- Mini-batch training
- Trade-offs in speed, memory and gradient noise
- Batch-size effects on optimization

Required Mastery

- Compare batch, stochastic and mini-batch updates in terms of computation and optimization behavior.

Module 14: Deep Learning Mid-Term Assessment

Scope / Lesson Components

- 14.1 Exam Instruction

- Quiz Module 14
- 14.1 Theoretical Mid-Term
- 14.2 Practical Mid-Term
- DL Mid-Term Exam

Detailed Knowledge Areas

- Consolidation of Modules 01–13
- Theoretical reasoning and numerical problems
- Practical PyTorch implementation
- Interpretation of training results

Module 15: Dataset and DataLoader in PyTorch

Scope / Lesson Components

- 15.1 Introduction
- 15.2 Problems in Previous Code
- 15.3 Implementing Batch Gradient Descent
- 15.4 Problem With Mini-Batch Implementation
- 15.5 Dataset and DataLoader Overview
- 15.6 Dataset and DataLoader Coding Example
- 15.7 Transformations and num_workers
- 15.8 Sampler
- 15.9 collate_fn
- 15.10 DataLoader Parameters
- 15.11 Improved Training Pipeline
- 15.12 Evaluation
- Quiz and practice

Detailed Knowledge Areas

- Dataset abstraction and reusable data interfaces
- `__len__` and `__getitem__`
- DataLoader and batching
- Transforms and preprocessing
- num_workers and data-loading performance
- Sampler and custom sampling
- collate_fn for batch construction
- Evaluation pipeline improvements

Required Mastery

- Implement a custom Dataset and DataLoader.
- Explain batch_size, shuffle, num_workers, sampler and collate_fn.

Module 16: ANN Implementation to GPU Training

Scope / Lesson Components

- 16.1 Module Introduction
- 16.2 Dataset
- 16.3 Architecture
- 16.4 Overall Workflow
- 16.5 Dataset and DataLoader
- 16.6 Define Model
- 16.7 Training Pipeline
- 16.8 Evaluation
- 16.9 CPU to GPU
- 16.10 Summary

Detailed Knowledge Areas

- End-to-end ANN project architecture
- Dataset/DataLoader integration
- Model definition and training
- Evaluation workflow
- CPU/GPU device management
- CUDA-aware tensor and model placement

Required Mastery

- Move model and tensors safely between CPU and GPU.
- Diagnose common device mismatch errors.

Module 17: Loss Functions and Training Challenges

Scope / Lesson Components

- 17.1 Introduction
- 17.2 Loss Function Introduction
- 17.3 Types of Loss Functions in Deep Learning
- 17.4 Mean Squared Error (MSE)
- 17.5 Mean Absolute Error (MAE)
- 17.6 Huber Loss – Part 1
- 17.7 Huber Loss – Part 2
- 17.8 Categorical Cross Entropy – Part 1
- 17.9 Categorical Cross Entropy – Part 2
- 17.10 Vanishing Gradient Problem – Part 1
- 17.11 Vanishing Gradient Problem – Part 2
- 17.12 Vanishing Gradient Problem – Part 3
- 17.13 Module Summary

Detailed Knowledge Areas

- MSE and squared-error sensitivity to outliers
- MAE and robustness

- Huber loss and transition between quadratic/linear behavior
- Categorical cross entropy for multiclass classification
- Logits, probabilities and classification loss
- Vanishing gradients and repeated chain-rule multiplication
- Sigmoid saturation and deep-network learning difficulty
- Mitigation through activations, initialization and architecture

Required Mastery

- Select MSE, MAE, Huber or cross entropy based on task.
- Explain the cause and consequences of vanishing gradients.

4. Seven-Day Detailed Study Schedule

DAY 1 — Perceptron, Decision Boundaries and Gradient Descent

Coverage: Modules 01–04

Session 1 | 1.5 h — Review deep-learning motivation, ML vs DL, biological/artificial neurons, perceptron architecture, weights, bias and activation.

Session 2 | 2.0 h — Perceptron learning rule, weight/bias updates, AND/OR gates, XOR limitation and implementation edge cases.

Session 3 | 1.0 h — Decision boundaries, linear separability and geometric interpretation of bias and weights.

Session 4 | 2.0 h — Gradient descent, derivatives, gradients, learning rate, loss landscape and parameter updates.

Session 5 | 2.0 h — Code a perceptron from scratch; solve AND/OR; visualize a decision boundary.

Session 6 | 0.5 h — Active recall and error review.

DAY 2 — Activation Functions, Losses, MLP and Forward Propagation

Coverage: Modules 05–06

Session 1 | 2.0 h — Sigmoid, derivative, saturation, ReLU, derivative, dying ReLU, Softmax, logits and probability interpretation.

Session 2 | 1.5 h — Perceptron loss and Binary Cross Entropy; numerical examples.

Session 3 | 1.5 h — MLP architecture, layers, neurons, parameters, matrix notation.

Session 4 | 2.0 h — Forward propagation using $Z = WX + b$ and $A = f(Z)$; solve a two-layer network by hand.

Session 5 | 1.5 h — Implement a small MLP forward pass using NumPy.

Session 6 | 0.5 h — Review formulas and explain the complete forward pipeline.

DAY 3 — Backpropagation and Chain Rule

Coverage: Module 08

Session 1 | 1.5 h — Backpropagation concept, forward pass, loss and backward pass.

Session 2 | 2.0 h — Partial derivatives and chain rule.

Session 3 | 1.5 h — Computational graphs and local/global gradients.

Session 4 | 2.0 h — Complete numerical backpropagation through a small network.

Session 5 | 1.5 h — Implement manual backpropagation with NumPy/Python.

Session 6 | 0.5 h — Review and explain why gradients flow backward.

DAY 4 — PyTorch Tensors and Autograd

Coverage: Modules 09–10

Session 1 | 2.0 h — Tensor creation, shape, dtype, scalar, element-wise, reduction and matrix operations.

Session 2 | 1.5 h — Comparison, special, in-place operations and tensor copying.

Session 3 | 2.0 h — `requires_grad`, computation graphs, `backward()` and `.grad`.

Session 4 | 1.5 h — Autograd with real examples and multiple variables.

Session 5 | 1.0 h — Gradient accumulation and clearing gradients.

Session 6 | 0.5 h — Implement a small autograd exercise and review.

DAY 5 — PyTorch Model Design and Training Dynamics

Coverage: Modules 11–13

Session 1 | 2.0 h — Complete training pipeline from preprocessing to evaluation.

Session 2 | 1.5 h — `nn.Module`, custom model classes, `forward()`, `Linear`, `ReLU` and `Sequential`.

Session 3 | 1.5 h — `torch.optim`, SGD, learning rate and parameter updates.

Session 4 | 2.0 h — Batch GD, SGD and Mini-Batch GD; batch-size effects.

Session 5 | 1.5 h — Build and train a complete ANN.

Session 6 | 0.5 h — Debug training-loop mistakes and review.

DAY 6 — Dataset, DataLoader, Evaluation and GPU

Coverage: Modules 15–16

Session 1 | 1.5 h — Custom Dataset, `__len__`, `__getitem__` and preprocessing.

Session 2 | 2.0 h — `DataLoader`, `batch_size`, `shuffle`, `num_workers`, `sampler`, `collate_fn` and transforms.

Session 3 | 2.0 h — Integrate Dataset/DataLoader with a complete training pipeline.

Session 4 | 1.0 h — Training/validation/test evaluation and performance metrics.

Session 5 | 1.5 h — CPU/GPU device management and CUDA workflow.

Session 6 | 0.5 h — Practice and troubleshooting.

DAY 7 — Loss Functions, Vanishing Gradient and Capstone

Coverage: Module 17 + cumulative review

Session 1 | 1.5 h — MSE, MAE and Huber loss with numerical comparison.

Session 2 | 1.5 h — Categorical Cross Entropy, logits and multiclass classification.

Session 3 | 1.5 h — Vanishing gradient, sigmoid saturation, deep chains and mitigation.

Session 4 | 3.5 h — Capstone: MNIST/Fashion-MNIST ANN using Dataset, DataLoader, model, loss, optimizer, training, evaluation and device management.

Session 5 | 1.0 h — Experiment with learning rate, batch size and activation function; record observations.

5. Mathematics Competency Checklist

- Weighted sum: $z = Wx + b$
- Activation transformation: $A = f(Z)$
- Gradient-descent update: $\theta_{\text{new}} = \theta_{\text{old}} - \eta \nabla L(\theta)$
- Perceptron learning rule and error-driven updates
- Sigmoid function and derivative
- ReLU function and derivative
- Softmax probability transformation
- Binary Cross Entropy
- Mean Squared Error, Mean Absolute Error and Huber loss
- Categorical Cross Entropy
- Partial derivatives and chain rule
- Layer-wise gradient propagation
- Relationship between computation graphs and derivatives

6. PyTorch Competency Checklist

- Create and manipulate tensors
- Understand shape and dtype
- Perform element-wise and matrix operations
- Use `requires_grad` and `autograd`
- Read and clear gradients
- Create custom `nn.Module` classes
- Use `nn.Linear`, `nn.ReLU` and `nn.Sequential`
- Define and use loss functions
- Configure `torch.optim`
- Write a robust training loop
- Implement custom `Dataset` and `DataLoader`
- Handle batching, shuffling, transforms and sampling
- Evaluate models on validation/test data
- Move tensors and models between CPU and GPU safely

7. Capstone Project Specification

Build an end-to-end ANN image-classification system using MNIST or Fashion-MNIST. The project must demonstrate the complete learning pipeline rather than only model training.

1. Load and inspect the dataset.
2. Preprocess and normalize input features.
3. Create a PyTorch `Dataset` and `DataLoader`.
4. Define an ANN using `nn.Module` or `Sequential`.
5. Select an appropriate output layer and classification loss.
6. Configure an optimizer and learning rate.
7. Implement the training loop with `zero_grad()`, forward pass, loss calculation, `backward()` and `optimizer.step()`.
8. Track training and validation loss/accuracy.
9. Evaluate on the test set.
10. Run at least three controlled experiments involving learning rate, batch size, activation or architecture.

11. Document observations, errors, fixes and final results.

8. Assessment and Evidence of Mastery

Component	Suggested Weight	Evidence
Conceptual understanding	20%	Oral/written explanations and short-answer questions
Mathematical reasoning	25%	Derivations and numerical forward/backward problems
PyTorch implementation	25%	Working tensor, autograd and training code
Practical debugging	10%	Diagnosis of shape, gradient, loss and device issues
Capstone project	20%	Reproducible ANN project and results

9. Final Competency Standard

At the end of the seven-day intensive program, the learner should be able to move independently from a supervised learning problem to a working neural-network solution: formulate the model, explain its mathematics, implement it in PyTorch, construct the data pipeline, train and evaluate it, interpret optimization behavior, and diagnose common training failures.

10. Recommended Daily Output

- 1 page of concise conceptual notes
- At least 2 mathematical worked examples
- At least 1 coding implementation or substantial modification
- At least 5 self-test questions
- One short explanation of the day's most difficult concept without notes
- Git/Notebook checkpoint containing working code